

1)

a. La clase instrumento debe de ser abstracta ya que contiene métodos abstractos y porque actúa como un 'borrador' para sus subclases, y esa es una de las cosas que la diferencia de una clase normal, las clases abstractas funcionan como un prototipo para sus subclases, mientras que una clase normal define una implementación completa para un objeto.

b. public class Piano extends Instrumento {

```
    public Piano(String tipo) {  
        super(tipo);  
    }  
    public void Tocar() {  
        System.out.println("Tocando Piano")  
    }  
    public void Afinar() {  
        System.out.println("Afinando Piano")  
    }  
}
```

}

c. Todas las líneas pueden ser ejecutadas sin problemas, es valido ya que tanto Guitarra como Saxofon son subclases de Instrumento y x declara una referencia de tipo Instrumento.

d.

Tocando Saxofon

Tocando Guitarra

2)

a. El método explotar esta definido como abstracto, pero al mismo tiempo tiene una implementación, por lo cual habría un error.

b. Son métodos normales con implementaciones que pueden ser utilizados por las subclases de ObjetoAstronomicoExtraSolar.

c. No, no se puede considerar como un error ya que las clases abstractas pueden tener tanto métodos normales como métodos abstractos, puede ser poco convencional, pero puede tener usos, como extender la clase y añadiendo métodos abstractos en el futuro.

d. El arreglo `oa` recibe objetos de tipo `ObjetoAstronomicoExtraSolar`, pero sus subclases también son objetos de tipo `ObjetoAstronomicoExtraSolar`, entonces el arreglo puede invocar los métodos específicos de cada subclase sin problema.

e. Porque en la línea 28 `oa[0]` empieza a apuntar al mismo objeto que estaba apuntando `oa[2]` que es Super Nova.

f. La clase `Estrella` hereda el método de `ObjetoAstronomicoEstraSolar`.

g. Genera un error debido a que los métodos sobrescritos no pueden reducir la visibilidad del método, ya que haciendo el método privado significaría que la clase padre u otras clases ya no tendrían acceso al método.

h. Porque esta heredando el método que su clase padre `Estrella` sobrescribió como un método normal. Si el método se definiera de nuevo habría sobreescritura de métodos.

i. El método `toString` es heredado de la clase padre `Object`, por lo tanto, imprimiría el nombre de la clase y su hashcode separados por un `@`.

j. Porque no se esta referenciando directamente a la clase `ObjetoAstronomicoExtraSolar`, sino, que esta asignando un objeto de tipo `Nova` (Que es subclase de `ObjetoAstronomicoExtraSolar`) a una referencia de ese tipo.

k. La línea B esta errónea debido a que esta instanciando a la clase `ObjetoAstronomicoExtraSolar` que es abstracta. Y la línea C seria valida ya que hay una relación de herencia que permite hacer la referencia.

l. La línea B esta correcta por lo mencionado en el punto anterior: "Y la línea C seria valida ya que hay una relación de herencia que permite hacer la referencia."

La línea C estaría errónea ya que a la hora de compilación el método `explotar()` no esta definido en la clase `ObjetoAstronomicoExtraSolar`.

m. Imprime `true` ya que la clase `Object` es una clase padre (Ya sea directa o indirectamente) de las clases creadas en Java.

El primer print imprimiría `false` ya que `null` no es instancia de ninguna clase.

El segundo print imprimiría `true` ya que al agregar `""` +, el resultado es un string, que si es una instancia de `Object`.

n. El constructor se puede agregar a la clase abstracta sin problema y serviría cada que una subclase es instanciada, su ID seria 4, pero el método `tipoCuerpo2()` seria llamado desde la subclase y no desde la clase padre, esto debido al polimorfismo.

o. La clase `EnanaBlanca` necesitaría implementar los métodos abstractos de las clase `Estrella` y `ObjetoAstronomicoExtraSolar` para que funcione correctamente.